



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

(深圳)

实验设计报告

开课学期: 2024 年秋季

课程名称: 操作系统

实验名称: XV6 与 UNIX 实用程序

实验性质: 课内实验

实验时间: 9.27 地点: T2210

学生班级: 5

学生学号: 220110515

学生姓名: 金正达

评阅教师: _____

报告成绩: _____

实验与创新实践教育中心印制

2024 年 9 月

一、 回答问题

1. 阅读 sleep.c, 回答下列问题

(1) 当用户在 xv6 的 shell 中, 输入了命令"sleep hello world\n", 请问在 sleep 程序里面, argc 的值是多少, argv 数组大小是多少?

argc 为 3

argv 数组的大小为 4 (包括最后的 NULL 指针)

(2) 请描述上述第一道小题 sleep 程序的 main 函数参数 argv 中的指针指向了哪些字符串, 它们的含义是什么?

argv[0] 指向 "sleep": 程序的名称

argv[1] 指向 "hello": 第一个命令行参数

argv[2] 指向 "world": 第二个命令行参数

argv[3] 是 NULL: 标志参数列表的结束

(3) 程序 sleep 使用了哪些系统调用?

sleep: 用于使进程休眠指定的时间

exit: 用于终止程序并返回状态码

2. 了解管道模型, 回答下列问题

(1) 简要说明在 pingpong 实验中, 你是怎么创建管道的? 结合 fork 系统调用说明你是怎么使用管道在父子进程之间传输数据的。

创建管道:

代码使用 pipe(p1) 和 pipe(p2) 创建了两个管道。

调用 fork() 创建子进程, 父进程和子进程都继承了这两个管道的文件描述符。

父子进程间的数据传输:

父进程:

关闭不需要的管道端: close(p1[0]) 和 close(p2[1])

向 p1 写入 "ping": write(p1[1], "ping", 4)

从 p2 读取响应: read(p2[0], &p_read, 4)

子进程:

关闭不需要的管道端: `close(p1[1])` 和 `close(p2[0])`

从 p1 读取消息: `read(p1[0], &c_read, 4)`

向 p2 写入 "pong": `write(p2[1], "pong", 4)`

这种设置实现了双向通信:

p1 用于父进程向子进程发送数据

p2 用于子进程向父进程发送数据

。

(2) 再次阅读[实验原理-管道](#)这一节中给出的关于 `wc` 命令的示例程序, 假设子进程没有关闭管道写端, 运行该程序后, 尝试描述会发生什么? 为什么在 Linux 环境下编译运行再执行 `ps` 指令会有如下结果?

```
C test.c > main()
1  #include <unistd.h>
2
3  int main() {
4      int p[2];          /* 存储管道的两个文件描述符 */
5      char *argv[2];
6      argv[0] = "wc";    /* 设置要执行的命令为"wc" */
7      argv[1] = 0;       /* 参数数组以NULL结尾, 表示没有更多参数 */
8      pipe(p);           /* 创建管道, p[0]是管道的读端, p[1]是管道的写端 */
9
10     if (fork() == 0) {
11         /* 子进程 */
12         close(0);        /* 关闭标准输入 (文件描述符0) */
13         dup(p[0]);       /* 复制管道的读端p[0], 让文件描述符0指向管道的读端 */
14         close(p[0]);     /* 关闭不再需要的管道读端 */
15         // close(p[1]);  /* 关闭不再需要的管道写端 */
16         execv("/bin/wc", argv); /* 执行"wc"程序 */
17     } else {
18         /* 父进程 */
19         close(p[0]);     /* 关闭管道的读端 */
20         write(p[1], "hello world\n", 12); /* 向管道写入"hello world\n" */
21         close(p[1]);     /* 关闭管道的写端, 表示写入完成 */
22     }
23     return 0;
24 }
```

```
● ~/os/xv6-oslab24-hitsz | on util ?4 gcc test.c -o test
● ~/os/xv6-oslab24-hitsz | on util ?4 ./test
● ~/os/xv6-oslab24-hitsz | on util ?4 ps
```

PID	TTY	TIME	CMD
44896	pts/6	00:00:01	zsh
44938	pts/6	00:00:00	gitstatusd-linu
46667	pts/6	00:00:00	wc
46724	pts/6	00:00:00	ps

```
○ ~/os/xv6-oslab24-hitsz | on util ?4
```

wc 程序通过标准输入读取数据，会在读到文件结束 (EOF) 时停止并输出结果。由于写端没有关闭，wc 程序会一直等待更多的输入，而不是结束执行。wc 程序没有结束，父进程在等待子进程结束时会被无限期阻塞。

没有关闭子进程写端，wc 没有退出，在等待更多的输入。

二、 实验详细设计

(1) pingpong 设计

创建管道：

程序开始时创建两个管道 (p1 和 p2)。p1 用于父进程向子进程发送消息，p2 用于子进程向父进程发送消息。

进程创建：

使用 fork() 创建一个子进程。fork() 之后，代码会在父进程和子进程中同时执行。

子进程逻辑：

关闭不需要的管道端：p1 的写端和 p2 的读端。

从 p1 读取消息（期望是“ping”）。

如果收到“ping”，则向 p2 写入“pong”。

打印接收到 ping 的消息，包括自己的 PID 和父进程的 PID。

关闭剩余的管道端并退出。

父进程逻辑：

关闭不需要的管道端：p1 的读端和 p2 的写端。

向 p1 写入“ping”消息。

从 p2 读取消息（期望是“pong”）。

等待子进程结束（使用 wait(0)）。

如果收到“pong”，打印接收到 pong 的消息，包括自己的 PID 和子进程的 PID。

关闭剩余的管道端并退出。

通信流程：

父进程发送“ping” -> 子进程接收“ping”

子进程发送“pong” -> 父进程接收“pong”

双方打印各自收到的消息。

(2) find 设计

主函数 (main)：

检查命令行参数，确保用户提供了目录和文件名。

调用 find 函数开始搜索过程。

查找函数 (find):

递归函数，用于遍历目录树。

主要步骤为：打开当前目录，读取目录内容，对每个条目进行处理，如果是文件且名称匹配，打印路径，如果是目录，递归调用 find。

路径处理:

使用 buf 数组来构建完整的文件路径。

检查路径长度以防止缓冲区溢出。

目录遍历:

使用 while 循环读取目录中的每个条目。

跳过 "." 和 ".." 目录以避免无限递归。

文件类型判断:

使用 stat 结构来获取文件类型信息。

根据文件类型（普通文件或目录）决定如何处理。

名称匹配:

使用 strcmp 函数比较文件名。

输出结果:

当找到匹配的文件或目录时，打印完整路径

(3) xv6 启动流程分析

```

1   b trap.c:55
2   c
3   p cpus[$tp]->proc->name
4   n
5   p cpus[$tp]->proc->name
6   da

```

1: 设置断点，在 trap.c 中 usertrap 函数中的 syscall() 函数调用，该处为 initcode 进程和 init 进程的分界；

2: 运行到断点处；

3: 打印此时的进程信息，为 initcode；

4: 步过 syscall 的调用，进入 init 进程；

5: 打印此时的进程信息，为 init;

三、实验结果截图

sleep:

```
(base) jzd@DESKTOP-75609C8:~/xv6-oslab24-hitsz$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
-bus.0
kernel/main.c:14
kernel/main.c:15      xv6 kernel is booting
kernel/main.c:16
kernel/main.c:36      hart 2 starting
kernel/main.c:36      hart 1 starting
init: starting sh
$ sleep 10
Nothing happens for a little while
$
```

pingpong:

```
(base) jzd@DESKTOP-75609C8:~/xv6-oslab24-hitsz$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -n
-bus.0
kernel/main.c:14
kernel/main.c:15      xv6 kernel is booting
kernel/main.c:16
kernel/main.c:36      hart 1 starting
kernel/main.c:36      hart 2 starting
init: starting sh
$ pingpong
4: received ping from pid 3
3: received pong from pid 4
$
```

find:

```
-bus.0
kernel/main.c:14
kernel/main.c:15      xv6 kernel is booting
kernel/main.c:16
kernel/main.c:36      hart 2 starting
kernel/main.c:36      hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
$
```

make grade:

```

== Test sleep, no arguments ==
$ make qemu-gdb
sleep, no arguments: OK (2.4s)
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (1.0s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (1.0s)
== Test pingpong lenient testing ==
$ make qemu-gdb
pingpong lenient testing: OK (0.9s)
== Test pingpong strict testing with changing pids ==
$ make qemu-gdb
pingpong strict testing with changing pids: OK (1.0s)
== Test find, in current directory and create a file ==
$ make qemu-gdb
find, in current directory and create a file: OK (1.0s)
== Test find, in current directory and create a dir ==
$ make qemu-gdb
find, in current directory and create a dir: OK (1.0s)
== Test find, find file recursive ==
$ make qemu-gdb
find, find file recursive: OK (1.2s)
== Test find, find dir recursive with no duplicates ==
$ make qemu-gdb
find, find dir recursive with no duplicates: OK (1.0s)
Score: 60/60

```

xv6 启动:

```

— Breakpoints —
[1] break at 0x0000000800029aa in kernel/trap.c:55 for trap.c:55 hit 1 time
— Expressions —
— History —
$$1 = "initcode\000\000\000\000\000\000\000"
$$0 = "init\000\000de\000\000\000\000\000\000"
— Memory —
— Registers —
zero 0x0000000000000000    ra 0x00000000080029b2    sp 0x0000003fffffdfe0
gp 0x0505050505050505    tp 0x0000000000000000    t0 0x0000000080002954
t1 0x80000000000087fff    t2 0x0505050505050505    fp 0x00000003fffffe000
s1 0x00000000080014d68    a0 0x0000000000000001    a1 0x0000000000000001
a2 0x00000000000001000    a3 0x0000000000000001    a4 0x0000000000000000
a5 0x00000000200000022    a6 0x00000000087f6f000    a7 0x0000000000000007
s2 0x0505050505050505    s3 0x0505050505050505    s4 0x0505050505050505
s5 0x0505050505050505    s6 0x0505050505050505    s7 0x0505050505050505
s8 0x0505050505050505    s9 0x0505050505050505    s10 0x0505050505050505
s11 0x0505050505050505    t3 0x0505050505050505    t4 0x0505050505050505
t5 0x0505050505050505    t6 0x0505050505050505    pc 0x00000000800029b2
— Source —
59     printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
60     printf("          sepc=%p stval=%p\n", r_sepc(), r_stval());
61     p->killed = 1;
62 }
63
64     if (p->killed) exit(-1);
65
66     // give up the CPU if this is a timer interrupt.
67     if (which_dev == 2) yield();
68
— Stack —
[0] from 0x00000000800029b2 in usertrap+94 at kernel/trap.c:64
[1] from 0x0505050505050505
— Threads —
[1] id 1 from 0x00000000800029b2 in usertrap+94 at kernel/trap.c:64
— Variables —
loc which_dev = 0, p = 0x80014d68 <proc>: {lock = {locked = 0,name = 0x80008208 "proc",c
pu = 0x0},state = RUNNING,
>>>

```